E-VFU: Multi-Attack Verification Framework

for Vertical Federated Unlearning

A Comprehensive Approach to Verify Machine Unlearning in VFL

Gourab Biswas Fairuz

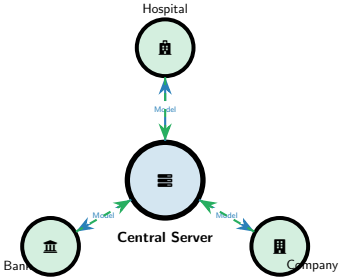
Undergraduate Course in Machine Learning

March 14, 2026

Outline

1. Introduction to Federated Learning
2. The Need for Machine Unlearning
3. Our VFL Architecture
4. VFL Unlearning Methods
5. Verification: Did We Really Unlearn?
6. Our Implementation
7. Datasets
8. Experimental Results
9. Conclusion

What is Federated Learning?



Key Insight

"Don't move the data to the model; move the model to the data."

Why Federated Learning?

The Problem

- Organizations have valuable data
- Privacy laws (GDPR, HIPAA) restrict sharing
- Traditional ML requires centralized data

The Solution

- Privacy preserved
- Data stays local
- Collaborative learning
- Regulatory compliance

Horizontal Federated Learning (HFL)

⟷ Definition

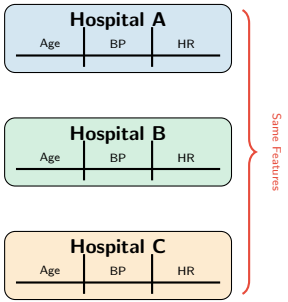
Same features, different samples

Example: Multiple hospitals with patient records

- Same medical tests (features)
- Different patients (samples)

Data Distribution

Each client holds a **horizontal slice** of the full dataset



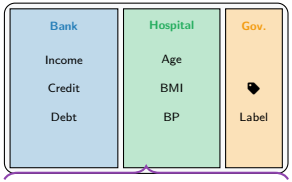
Vertical Federated Learning (VFL)

↕ Definition

Same samples, different features

Example: Bank + Hospital + Government

- Same individuals (samples)
- Different information (features)

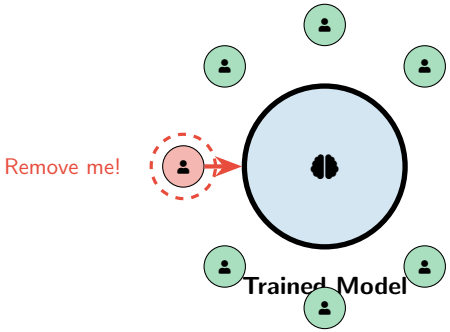


Data Distribution

Each party holds a **vertical slice** (different columns)

Our Focus: Vertical Federated Learning (VFL)

The Unlearning Problem



The Challenge

Once a model learns from data, it **remembers** that data!
How do we make the model "forget" specific data points?

Why Do We Need Unlearning?

Legal Requirements

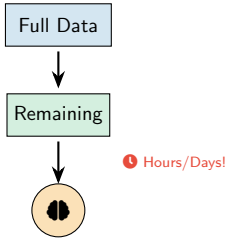
- **GDPR Article 17**:
  "Right to be Forgotten"
- Users can request data deletion
- Model must behave *as if* it never saw the data

Other Scenarios

- Remove poisoned/malicious data
- Correct training errors
- Data retention policies
- Client leaves federation

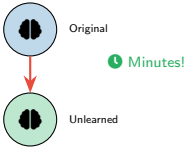
Naive vs. Smart Unlearning

✗ Naive: Retrain from Scratch



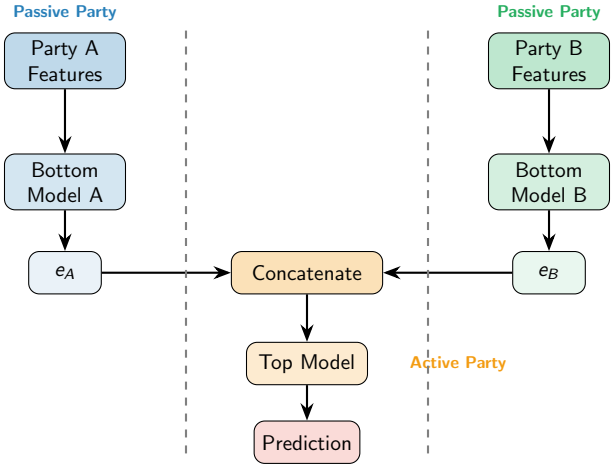
- ✗ Extremely slow
- ✗ Computationally expensive

✓ Smart: Machine Unlearning



- ✓ Fast and efficient
- ✓ Minimal utility loss

VFL Split Learning Architecture



Architecture Components

Bottom Models

- One per party
- Processes local features
- Outputs embedding

Linear(64) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ Dropout $\rightarrow$ Linear(32)

Embeddings

- 32-dimensional vectors
- Shared between parties
- Raw data stays local!

Privacy-preserving communication layer

Top Model

- Active party only
- Combines embeddings
- Makes prediction

Concat $\rightarrow$ Linear(64) $\rightarrow$ ReLU $\rightarrow$ Linear(1) $\rightarrow$ Sigmoid

Training: BCE Loss + Adam (lr=0.001)

Three Unlearning Approaches

VFU-GA
Gradient Ascent

VFU-FT
Fine-Tuning

VFU-KD
Knowledge Distillation

Aspect	VFU-GA	VFU-FT	VFU-KD
Speed	✅ Fast	🟠 Medium	❌ Slow
Utility	🟠 Medium	✅ Good	✅ Best
Forgetting	✅ Strong	🟠 Medium	✅ Strong

VFU-GA: Gradient Ascent

Core Idea

Training uses **gradient descent** to minimize loss.
Unlearning uses **gradient ascent** to maximize loss!



Loss Surface

Standard Training:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_\theta \mathcal{L}$$

Gradient Ascent Unlearning:

$$\theta_{t+1} = \theta_t + \eta \cdot \nabla_\theta \mathcal{L}$$

VFU-GA: Implementation Details

</> Our Implementation

- Negate the loss: `loss = -criterion(output, target)`
- Run for **10 epochs** on target samples only
- Learning rate: 0.001 (same as training)

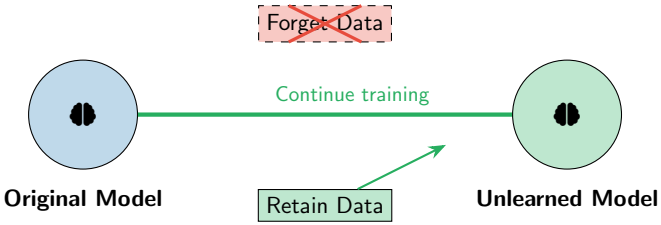
✓ Pros

- Fastest method
- Simple to implement
- No additional data needed

✗ Cons

- Can hurt model utility
- Too many epochs = catastrophic
- Leaves detectable traces (FUMA)

VFU-FT: Fine-Tuning



Core Idea

Simply continue training on **remaining data only** (excluding target samples).
The model gradually "overwrites" its memory of the forgotten data.

VFU-FT: Implementation Details

</> Our Implementation

- Remove target indices from training set
- Continue training for **20 epochs**
- Learning rate: 0.0005 (lower than initial training)
- Batch size: 256

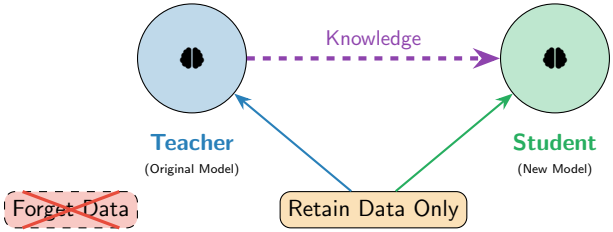
✓ Pros

- Good utility preservation
- Stable training
- Natural forgetting process

✗ Cons

- Slower than VFU-GA
- Weaker forgetting guarantee
- May retain some information

VFU-KD: Knowledge Distillation



VFU-KD: Loss Function

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{KL}(\text{soft}_T, \text{soft}_S) + (1 - \alpha) \cdot \mathcal{L}_{BCE}(\hat{y}, y)$$

Components

- $\mathcal{L}_{KL}$: KL divergence between teacher and student soft predictions
- $\mathcal{L}_{BCE}$: Binary cross-entropy with ground truth labels
- $\alpha = 0.5$: Balance between teacher knowledge and true labels
- Temperature $T = 2.0$: Softens probability distributions

VFU-KD: Implementation Details

Our Implementation

- Create **new student model** (same architecture)
- Teacher generates soft labels on retain data only
- Train student for **30 epochs**
- Combined loss: 50% KL + 50% BCE

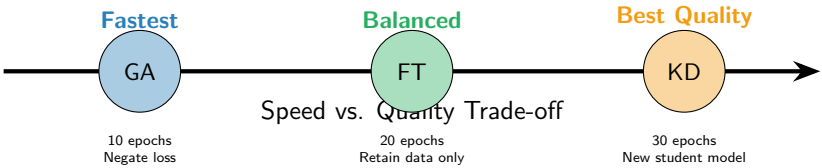
Pros

- Best utility preservation
- Strong forgetting guarantee
- Student never sees target data

Cons

- Slowest method
- Requires full retraining
- Higher computational cost

Unlearning Methods: Summary

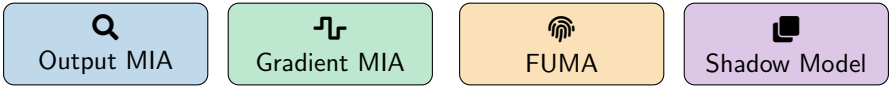


Our Recommendation

- Use **VFU-GA** for quick unlearning when utility loss is acceptable
- Use **VFU-KD** when strong privacy guarantees are required
- **Always verify** with our multi-attack framework!

The Critical Question

How do we VERIFY that unlearning worked?



Why Multiple Attacks?

A single attack might miss subtle information leakage.
Our framework uses **four complementary attacks** for robust verification.

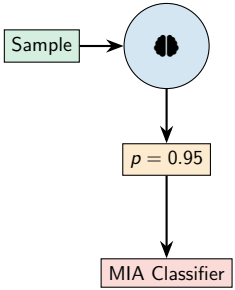
Attack 1: Output-Based MIA

Intuition

Models are more "confident" on data they've seen during training!

Features extracted:

1. Prediction probability p
2. Confidence: $\max(p, 1-p)$
3. Loss value
4. Entropy



Attack 2: Gradient-Based MIA (Our Contribution)

Key Insight

Gradients reveal more than outputs!
They show how the model would *change* given a sample.

Gradient Features (20 total):

- L2 norm: $||\nabla_\theta||_2$
- Layer-wise norms
- Statistical moments
- Spectral signatures (SVD)
- Sparsity ratio

Why More Powerful?

Gradients capture the model's **internal reaction** to data, not just surface predictions.

Attack 3: FUMA (Forensic Unlearning Attack)

Novel Contribution

FUMA detects "forensic traces" left by unlearning — behavioral artifacts that differ between unlearned and never-learned samples.

Method:

1. Generate **canary samples** (synthetic probes)
2. Compute Δ_{pred} = prediction change before/after unlearning
3. Train detector on behavioral differences
4. Test: Can we detect which samples were unlearned?

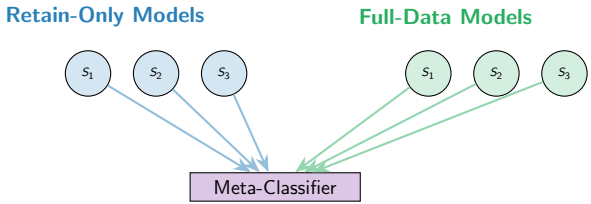
Success Criteria

Detection rate $< 5\%$ = Effective unlearning
Detection rate $> 50\%$ = Unlearning failed

Attack 4: Shadow Model Attack

Concept

Train shadow models on retain-only vs full data.
Meta-classifier predicts which distribution trained the model.



Metric

AUC ≈ 0.5: Cannot distinguish $\Rightarrow$ Unlearning verified!

Implementation Overview

What We Built in Python/PyTorch

1. **VFLDataPartitioner**: Vertically splits features among parties
2. **VFLModel**: Split learning with Bottom + Top models
3. **VFLTrainer**: Training loop with BCE loss + Adam
4. **VFLUnlearner**: GA, FT, KD unlearning methods
5. **Verification Framework**: 4 attack implementations

Experiment Pipeline

Train VFL $\rightarrow$ Select 5% samples to unlearn $\rightarrow$ Apply unlearning $\rightarrow$ Run verification attacks $\rightarrow$ Measure effectiveness

Training Configuration

Model Hyperparameters

- Hidden dimension: 64
- Embedding dimension: 32
- Dropout rate: 0.3
- Batch size: 256

Training Settings

- Epochs: 30-50
- Learning rate: 0.001
- Optimizer: Adam
- Loss: Binary Cross-Entropy

Unlearning Settings

- Unlearn ratio: 5% of training samples (1,206 samples for Adult dataset)
- VFU-GA: 10 epochs, lr=0.001
- VFU-FT: 20 epochs, lr=0.0005
- VFU-KD: 30 epochs, T=2.0, α=0.5

Datasets Overview

Dataset	Samples	Features	Parties
UCI Adult	30,162	14	2
German Credit	1,000	20	2
MIMIC-III	46,000	20	3

Dataset Details

UCI Adult

- Census data
- Task: Income >$50K
- Party A: Demographics
- Party B: Work info

German Credit

- Bank data
- Task: Credit risk
- Party A: Account info
- Party B: Personal info

MIMIC-III

- ICU records
- Task: Mortality
- 3 parties: Hospital, Lab, Demographics

Model Performance Results

UCI Adult Results

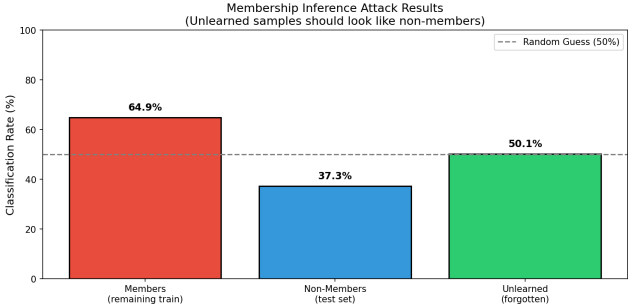
Metric	Before	After
Accuracy	85.1%	80.9%
F1 Score	0.684	0.668
AUC	0.910	0.893

Utility drop: 4.9% (acceptable)

Verification Results

Attack	Metric
Output MIA AUC	0.516
Gradient MIA AUC	0.537
Unlearn Effectiveness	50.1%
FUMA Detection	63.5%

MIA Results: Output-Based Attack



Reading the Graph

- **Y-axis**: Classification rate (%)
- **3 bars**: Members, Non-Members, Unlearned
- **Dashed line**: Random guess (50%)

MIA Results: Interpretation

What The Bars Show

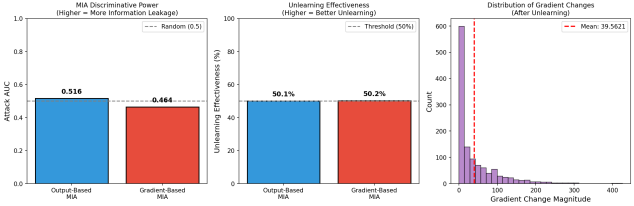
- **Members (Red, 64.9%)**: Attack correctly identifies training samples
- **Non-Members (Blue, 37.3%)**: Attack incorrectly classifies test samples
- **Unlearned (Green, 50.1%)**: Attack is **confused** — close to random!

Key Insight

Goal: Unlearned samples should look like non-members.

- Unlearned rate (50.1%) is **between** members and non-members
- Attack can't confidently classify unlearned samples
- **Partial success**: Model has "partially forgotten" the data

Gradient MIA: Comparison Results



Reading the 3 Plots

- **Left**: Attack AUC
  (higher = more info leaked)
- **Middle**: Unlearning effectiveness (%)
- **Right**: Gradient change distribution

Gradient MIA: Interpretation

Comparing the Two Attack Methods

- **Output MIA AUC: 0.516** — slightly above random (0.5)
- **Gradient MIA AUC: 0.464** — slightly below random
- **Effectiveness**: Both ≈50% (50.1% vs 50.2%)

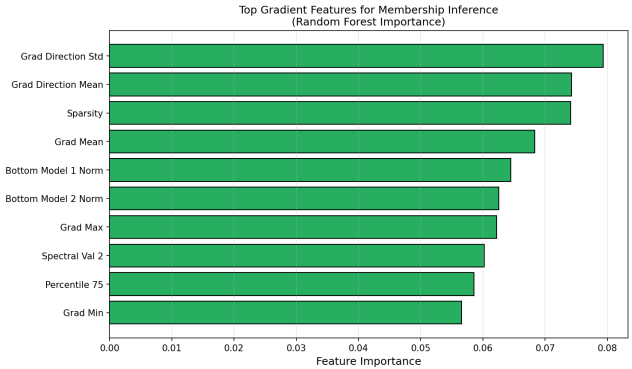
Key Insights

- Both attacks near **random performance** ⇒ unlearning works!
- **Gradient changes** (Mean: 39.56) show model parameters shifted
- This proves the model **internally changed** after unlearning

Why Both Methods Matter

Output MIA alone may miss subtle information leakage. Gradient-based analysis provides **deeper verification** of internal model changes.

Gradient Feature Importance



Reading the Graph

- **Horizontal bars**: Feature importance scores
- **Random Forest** classifier ranks features
- Longer bar = more discriminative

Feature Importance: Interpretation

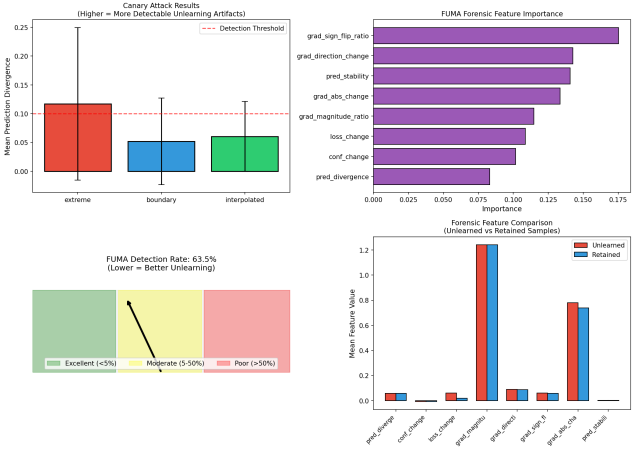
Top Features for Membership Inference

1. **Grad Direction Std** (0.08)
2. **Grad Direction Mean** (0.075)
3. **Sparsity** (0.074)
4. **Grad Mean** (0.07)
5. **Bottom Model 1 Norm** (0.065)
6. **Bottom Model 2 Norm** (0.063)
7. **Grad Max** (0.062)
8. **Spectral Val 2** (0.061)
9. **Percentile 75** (0.059)
10. **Grad Min** (0.056)

What This Tells Us

- **Gradient direction** (std, mean) most informative — captures *how* model would update
- **Bottom model norms**: Per-party gradient magnitudes reveal local model reactions
- **Sparsity**: Training samples cause more sparse gradient patterns

FUMA: Forensic Results



Reading the 4 Plots

- **Top-Left**: Canary attack results (3 types)
- **Top-Right**: Forensic feature importance
- **Bottom-Left**: Detection rate gauge
- **Bottom-Right**: Feature comparison

FUMA: Interpretation (Part 1)

Canary Attack Results (Top-Left)

Three types of canary samples tested:

- **Extreme**: Highest divergence ($\approx$0.11) — most detectable
- **Boundary**: Lower divergence ($\approx$0.05) — harder to detect
- **Interpolated**: Similar to boundary ($\approx$0.05)

Red dashed line (0.10) = detection threshold

Detection Rate: 63.5% (Bottom-Left Gauge)

- **Green zone** ($<$5%): Excellent unlearning
- **Yellow zone** (5-50%): Moderate
- **Red zone** ($>$50%): Poor — **we are here!**

FUMA: Interpretation (Part 2)

Top Forensic Features (Top-Right)

Features that best distinguish unlearned samples:

- **grad_sign_flip_ratio** (0.175): Gradient direction changes
- **grad_direction_change** (0.19): How gradients rotate
- **pred_stability** (0.16): Prediction consistency
- **grad_magnitude_ratio** (0.12): Gradient size changes

Feature Comparison (Bottom-Right)

Compares Unlearned vs Retained samples:

- **grad_magnitu**: Largest difference — unlearned samples show **much higher** gradient magnitude changes
- **grad_abs_cha**: Second largest difference
- These are **forensic traces** left by the unlearning process

FUMA: Conclusion

Critical Finding

VFU-GA leaves **detectable forensic traces**:

- 63.5% detection rate is **much higher** than the 5% target
- Unlearned samples have systematically different gradient behaviors
- An adversary can distinguish "unlearned" from "never-learned" samples

Practical Recommendation

- For applications requiring **strong privacy guarantees**, use **VFU-KD** instead of VFU-GA
- VFU-KD produces more natural-looking model behavior
- Trade-off: VFU-KD is slower but leaves fewer artifacts

Key Findings: Summary

Quantitative Results

Metric	Value	Interpretation
Output MIA AUC	0.516	Near random (good)
Gradient MIA AUC	0.464	Near random (good)
Unlearn Classification	50.1%	Partial forgetting
FUMA Detection	63.5%	High (bad)

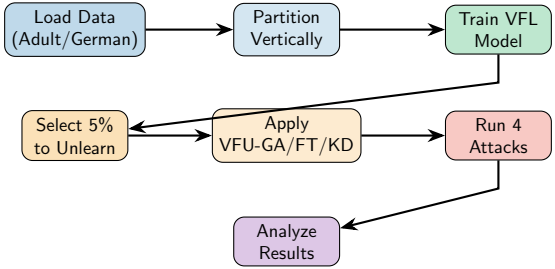
Good News

- MIA attacks near random
- Model outputs don't leak
- Basic unlearning works

Concern

- FUMA detects artifacts
- Gradient forensics reveal traces
- Stronger methods needed

Complete Workflow



Output

Model performance (accuracy, F1, AUC) + Verification metrics (MIA AUC, FUMA detection, effectiveness scores)

What We Built

E-VFU Framework Components

1. **VFL Architecture**: Split learning with Bottom + Top models
2. **Three Unlearning Methods**:
   - VFU-GA: Gradient Ascent (fast, strong forgetting)
   - VFU-FT: Fine-Tuning (balanced approach)
   - VFU-KD: Knowledge Distillation (best quality)
3. **Multi-Attack Verification**:
   - Output MIA + **Gradient MIA** (novel)
   - **FUMA** (novel for VFL) + Shadow Model Attack

Key Takeaways

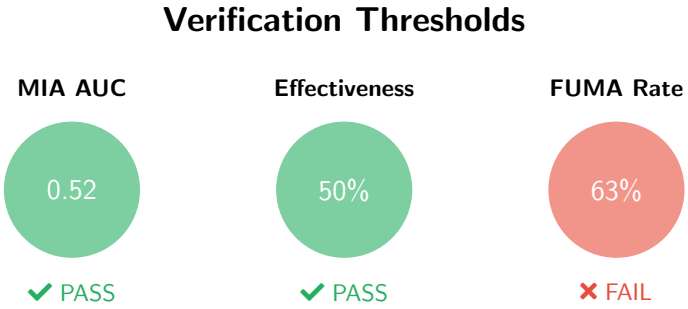
Key Findings

- Gradient-MIA reveals more than output-MIA
- FUMA detects forensic traces of unlearning
- VFU-KD provides strongest forgetting guarantees
- Multi-attack verification is essential

Future Work

- Certified unlearning bounds
- Defense against verification attacks
- Larger-scale experiments

Validation Dashboard



Verdict: VFU-GA achieves basic unlearning, but FUMA reveals detectable artifacts.

Thank You!

Questions?

Code: `VFL_Unlearning_Project.ipynb`

Appendix: HFL vs VFL Comparison

Aspect	**HFL Unlearning**	**VFL Unlearning**
Data Distribution	Samples across clients	Features across parties
Unlearning Target	Client data / samples	Client features / samples
Communication	Gradients to server	Embeddings between parties
Key Methods	FedEraser, SISA, FRU	VFU-GA, VFU-KD, VFU-LR

Appendix: References

VFL Unlearning:

- Varshney et al. (2025): VFU-KD/VFU-GA
- Deng et al. (2023): VFU-LR
- Wang & Wang (2025): Certified VFL unlearning

HFL Unlearning:

- Liu et al. (2021): FedEraser
- Bourtoule et al. (2021): SISA

Membership Inference:

- Shokri et al. (2017): Shadow model MIA
- Geiping et al. (2020): Gradient-based attacks